

Find
the
Runtime

```
void functionA(int n)
{
    for (int i = 1; i < n; i *= 2)
    {
        step++;
    }
}
```

$i = 0, 1, 2, 3, 4, \dots$
 $i = 1, 2, 4, 8, 16, 32, \dots$
 $O(\log n)$

```
void functionB(int n)
{
    for (int i = 0; i < n; i++) —  $O(n)$ 
    {
        for (int j = 0; j < n*n; ++j) —  $O(n^2)$ 
        {
            for (int k = 0; k < j; ++k) —  $O(n^2)$ 
            {
                step++; —  $O(1)$ 
            }
        }
    }
}
```

$$O(n^4) = O(n) * O(n^2) * O(n^2)$$

for () — $O(n)$
 $\{$
 $\}$

for () — $O(n^2)$

$$(n^4) = O(n) + O(n^2)$$

$\#$ Multiply when using nested loops

$\#$ Add when doing things in sequence

```

// Problem:
// Complete recBinarySearch
// Find it's recursive equation
// Build recursive tree
// Use tree to find asymptotic notation
int recBinarySearch(vector<int> &V, int start, int end, double key)
{
    if (start > end)
        return -1;
    mid = (end + start) / 2;
    int mid = start + (end - start) / 2;

    if (key == V[mid])
        return mid;
    else if (key < V[mid])
        return recBinarySearch(V, start, mid - 1, key);
    else if (key > V[mid])
        return recBinarySearch(V, mid + 1, end, key);
}

```

Recursive Equation: $T(n) = T(\frac{n}{2}) + c$

Base Case: $T(1) = c$

Next Steps:

- 1) Build tree w/ everything that is not recursive
- 2) Do next recursive call
- 3) Do this until you see a pattern
- 4) Set value of base case to pattern
- 5) Solve for i
- 6) Do a summation
- 7) Multiply top of tree of (i+1)

	Recursive Call	Tree	Row Sum	# of items in row
$T(n) = T(\frac{n}{2}) + c$	$T(n)$	$\textcircled{c}^n$	c	$1 = 2^0$
$T(\frac{n}{2}) = T(\frac{n/2}{2}) + c$	$T(\frac{n}{2})$	$\textcircled{c}^{\frac{n}{2}} \quad \textcircled{c}^{\frac{n}{2}}$	c	$2 = 2^1$
$T(\frac{n}{2^2}) = T(\frac{n/2^2}{2}) + c$	$T(\frac{n}{2^2})$	$\begin{matrix} c & c & c & c \\ / & \backslash & / & \backslash \end{matrix}$	c	$4 = 2^2$
	$T(\frac{n}{2^3})$	$\begin{matrix} c & c & c & c & c & c & c & c \\ / & \backslash & / & \backslash & / & \backslash & / & \backslash \end{matrix}$	c	$8 = 2^3$
	$\downarrow$	$\vdots$	$\downarrow$	$\downarrow$
	$T(\frac{n}{2^i})$		c	2^i

When to stop?

Base Case = $T(1) = c$

$$T\left(\frac{n}{2^i}\right) = 1$$

$$\frac{n}{2^i} = 1$$

$$n = 2^i$$

$$\log_2 n = i$$

$$\sum_{i=0}^{\log_2 n} c = \underbrace{c + c + c + c}_{\log_2 n}$$

$$c \sum_{i=0}^{\log_2 n} 1 = c \underbrace{(1 + 1 + 1 + 1)}_{\log_2 n}$$

Total runtime: work per level $\times$ # of levels

$$c \times (\log_2 n + 1)$$

$$\log_2 n + c = \underline{\underline{O(\log n)}}$$